Slip 01

Write a program to implement employee information in a file and perform the file operations on it.

ReadEmployee.java

```
package Slip1_1;
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;
public class ReadEmployee {
   public static void main(String[] args) {
      try {
         File file = new File("employee.txt");
         Scanner sc = new Scanner(file);
         System.out.println("Employee Details:\n");
         while (sc.hasNextLine()) {
            System.out.println(sc.nextLine());
         }
         sc.close();
      } catch (FileNotFoundException e) {
         System.out.println("File not found.");
      }}}
```

WriteEmployee.java

```
package Slip1_1;
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;
public class WriteEmployee {
   public static void main(String[] var0) {
      Scanner var1 = new Scanner(System.in);
      try {
         FileWriter var2 = new FileWriter("employee.txt");
```

```
            System.out.print("Enter Employee ID: ");
            int var3 = var1.nextInt();
            var1.nextLine();
            System.out.print("Enter Employee Name: ");
            String var4 = var1.nextLine();
            System.out.print("Enter Employee Salary: ");
            double var5 = var1.nextDouble();
            var2.write("ID: " + var3 + "\n");
            var2.write("Name: " + var4 + "\n");
            var2.write("Salary: " + var5 + "\n");
            var2.write("----------------------\n");
            var2.close();
            System.out.println("Employee data written successfully.");
        } catch (IOException var7) {
            System.out.println("Error: " + var7);
        }
        var1.close();
    }}
```

DeleteEmployee

```
package Slip1_1;
import java.io.*;
public class DeleteEmployee{
    public static void main(String[] args){
        File myFile = new File("employee.txt");
        if(myFile.delete())
        {
            System.out.println("Employee Record Deleted : " + myFile.getName());
        }
        else {
            System.out.println("Employee Record not deleted !");
        }}}
```

OR

Develop a Java application that connects to a MySQL database to manage student information. Implement CRUD operations for student records using JDBC. Use prepared statements to handle SQL queries securely and ensure proper transaction management.

```
import java.sql.*;
import java.util.Scanner;

public class StudentRecord {
   static final String URL = "jdbc:mysql://localhost:3306/slips";
   static final String USER = "root";
   static final String PASSWORD = "Shubham.k";

   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      try {
         Connection con = DriverManager.getConnection(URL, USER, PASSWORD);
         con.setAutoCommit(false);

         int choice;
         do {
            System.out.println("\n===== STUDENT CRUD OPERATIONS =====");
            System.out.println("1. Insert Student");
            System.out.println("2. Display Students");
            System.out.println("3. Update Student");
            System.out.println("4. Delete Student");
            System.out.println("5. Exit");
            System.out.print("Enter Choice: ");
            choice = sc.nextInt();

            switch (choice) {
               case 1:
                  System.out.print("Enter Student ID: ");
                  int id = sc.nextInt();
                  sc.nextLine();

                  System.out.print("Enter Student Name: ");
                  String name = sc.nextLine();
                  System.out.print("Enter Course: ");
                  String course = sc.nextLine();
                  String insertQuery ="INSERT INTO slip1_2 (id,name,course) VALUES(?,?,?)";
                  PreparedStatement ps1 =con.prepareStatement(insertQuery);
                  ps1.setInt(1, id);
```

```
    ps1.setString(2, name);
    ps1.setString(3, course);
    ps1.executeUpdate();
    con.commit();
    System.out.println("Student Inserted Successfully.");
    break;

  case 2:
    String selectQuery = "SELECT * FROM slip1_2";
    PreparedStatement ps2 = con.prepareStatement(selectQuery);
    ResultSet rs = ps2.executeQuery();

    System.out.println("\n--- Student Records ---");
    while (rs.next()) {
      System.out.println(
          rs.getInt("id") + "  " +
              rs.getString("name") + "  " +
              rs.getString("course")
      ); }
    break;
  case 3:
    System.out.print("Enter Student ID to Update: ");
    int updateId = sc.nextInt();
    sc.nextLine();

    System.out.print("Enter New Name: ");
    String newName = sc.nextLine();

    String updateQuery = "UPDATE slip1_2 SET name=? WHERE id=?";

    PreparedStatement ps3 =  con.prepareStatement(updateQuery);

    ps3.setString(1, newName);
    ps3.setInt(2, updateId);

    ps3.executeUpdate();
    con.commit();
    System.out.println("Student Updated Successfully.");
    break;
  case 4:
    System.out.print("Enter Student ID to Delete: ");
    int deleteId = sc.nextInt();

    String deleteQuery ="DELETE FROM slip1_2 WHERE id=?";
    PreparedStatement ps4 =con.prepareStatement(deleteQuery);

    ps4.setInt(1, deleteId);
```

```
                ps4.executeUpdate();
                con.commit();

                System.out.println("Student Deleted Successfully.");
                break;
            case 5:
                System.out.println("Program Ended.");
                break;
            default:
                System.out.println("Invalid Choice.");
        }
    } while (choice != 5);

} catch (Exception e) {
    System.out.println(e);
}}}
```

Slip 02

Create a hierarchy of Employee, Manager, Sales Manager, they should have the following functionality:

A. Employee: Display name, date of birth and id of Employee.

B. Manager: Display all above information with salary drawn.

C. Sales Manager: Display all above information and commission if applicable.

```
package Slip2_1;
class Emp{
   String name;
   String dob;
   int id;
   Emp(String name,String dob,int id){
      this.name=name;
      this.dob=dob;
      this.id=id;
   }
   void display(){
      System.out.println("Employee name is:"+name);
      System.out.println("Employee birth date:"+dob);
      System.out.println("Employee id is:"+id);
```

```
    }}
class Manager extends Emp{
    double salary;
    Manager(String name,String dob,int id,double salary){
        super(name,dob,id);
        this.salary=salary;
    }
    void display(){
        super.display();
        System.out.println("salary is:"+salary);
    }}
class SalesManager extends Manager
{
    double commission;
    SalesManager(String name,String dob,int id,double salary,double commission)
    {
        super(name,dob,id,salary);
        this.commission=commission;
    }
    void display(){
        super.display();
        System.out.println("Commission is:"+commission);
    }}
public class Employee{
    public static void main(String[] args){
        System.out.println("Employee details are");
        Emp emp=new Emp("Sk","18-5-2004",1);
        emp.display();
        System.out.println("Manager details are");
        Manager mrg=new Manager("Ak","19-4-2004",2,50000);
        mrg.display();
```

```
        System.out.println("Sales Manager details are");
        SalesManager sm=new SalesManager("Pk","20-6-2004",3,60000,5000);
        sm.display();
    } }
```

OR

Write a program to demonstrate how the process tasks asynchronously in a Spring Boot application.

```
/* 1.go to start.spring.io
2.download file
3.open the file in intellij and run the application
4.go to src/main/java/com/example/demo/DemoApplication.java
5.create class AsyncService,AsyncController,AsyncDemoApplication */

/* ADD code in AsyncDemoApplication.java */
package com.example.firstdemo;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.scheduling.annotation.EnableAsync;
@SpringBootApplication
@EnableAsync
public class AsyncDemoApplication {
    public static void main (String[] args) {
        SpringApplication.run(AsyncDemoApplication.class, args);
    }}

/* Add code in AsyncService.java */
package com.example.firstdemo;
import org.springframework.scheduling.annotation.Async;
import org.springframework.stereotype.Service;
@Service
public class AsyncService {
```

```
    @Async
    public void processTask() {
        try {
            System.out.println("Task started on thread: " +
                    Thread.currentThread().getName());
            Thread.sleep(5000); // simulate long process (5 seconds)
            System.out.println("Task completed on thread:" +
                    Thread.currentThread().getName());
        } catch (InterruptedException e) {
            e.printStackTrace();
        } }}

/* Add code in AsyncController.java */
package com.example.firstdemo;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
@RestController
public class AsyncController {
    @Autowired
    private AsyncService asyncService;
    @GetMapping("/run")
    public String runTask() {
        asyncService.processTask();
        return "Task triggered successfully";
    }}
```

Slip 03

Create a Java program to manage employee payroll using OOP concepts. Include functionalities such as employee creation, salary calculation, bonus allocation, and payslip generation. Use inheritance for different employee types (e.g., full-time,

part-time).

```
*import java.util.Scanner;
class Employee {
   int id;
   String name;
   double salary;
   Employee(int id, String name) {
      this.id = id;
      this.name = name;
   }
   void calculateSalary() {
   }
   void generatePayslip() {
      System.out.println("\n===== PAYSLIP =====");
      System.out.println("Employee ID : " + id);
      System.out.println("Employee Name : " + name);
      System.out.println("Total Salary : " + salary);
   }}
class FullTimeEmployee extends Employee {
   double monthlySalary;
   double bonus;
   FullTimeEmployee(int id, String name,
                  double monthlySalary, double bonus) {
      super(id, name);
      this.monthlySalary = monthlySalary;
      this.bonus = bonus;
   }
   void calculateSalary() {
      salary = monthlySalary + bonus;
   }}
class PartTimeEmployee extends Employee {
```

```
    int hoursWorked;
    double ratePerHour;
    PartTimeEmployee(int id, String name,int hoursWorked, double ratePerHour) {
        super(id, name);
        this.hoursWorked = hoursWorked;
        this.ratePerHour = ratePerHour;
    }
    void calculateSalary() {
        salary = hoursWorked * ratePerHour;
    }}
public class EmployeePayroll {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter Full-Time Employee Details");
        System.out.print("Enter ID: ");
        int id1 = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter Name: ");
        String name1 = sc.nextLine();
        System.out.print("Enter Monthly Salary: ");
        double msalary = sc.nextDouble();
        System.out.print("Enter Bonus: ");
        double bonus = sc.nextDouble();
        FullTimeEmployee emp1 =new FullTimeEmployee(id1, name1, msalary, bonus);

        emp1.calculateSalary();
        emp1.generatePayslip();
        System.out.println("\nEnter Part-Time Employee Details");
```

```
        System.out.print("Enter ID: ");
        int id2 = sc.nextInt();
        sc.nextLine();
        System.out.print("Enter Name: ");
        String name2 = sc.nextLine();

        System.out.print("Enter Hours Worked: ");
        int hours = sc.nextInt();
        System.out.print("Enter Rate Per Hour: ");
        double rate = sc.nextDouble();
        PartTimeEmployee emp2 =new PartTimeEmployee(id2, name2, hours, rate);
        emp2.calculateSalary();
        emp2.generatePayslip();
        sc.close();
    }}
```

OR

Write a program to design an admission enquiry for MCA student form using Swing.

```
package Slip3_2;

/*Write a program to design an admission enquiry for MCA student form using
Swing.*/


import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class Admission extends JFrame implements ActionListener
{
    JLabel l1, l2, l3, l4, l5, l6;
    JTextField t1, t2, t3;
    JRadioButton r1, r2;
    JComboBox c1;
```

```
JButton b1;

Admission()
{
   setTitle("MCA Admission Enquiry Form");

   l1 = new JLabel("Student Name:");
   l2 = new JLabel("Mobile Number:");
   l3 = new JLabel("Email ID:");
   l4 = new JLabel("Gender:");
   l5 = new JLabel("City:");
   l6 = new JLabel("Course:");

   t1 = new JTextField();
   t2 = new JTextField();
   t3 = new JTextField();

   r1 = new JRadioButton("Male");
   r2 = new JRadioButton("Female");

   ButtonGroup bg = new ButtonGroup();
   bg.add(r1);
   bg.add(r2);

   String course[] = {"MCA", "MBA", "MSc IT"};
   c1 = new JComboBox(course);

   b1 = new JButton("Submit");

   setLayout(null);

   l1.setBounds(50, 50, 120, 30);
   t1.setBounds(180, 50, 150, 30);

   l2.setBounds(50, 100, 120, 30);
   t2.setBounds(180, 100, 150, 30);

   l3.setBounds(50, 150, 120, 30);
   t3.setBounds(180, 150, 150, 30);

   l4.setBounds(50, 200, 120, 30);
   r1.setBounds(180, 200, 80, 30);
   r2.setBounds(260, 200, 80, 30);

   l5.setBounds(50, 250, 120, 30);

   String city[] = {"Mumbai", "Pune", "Nashik"};
```

```
   JComboBox cityBox = new JComboBox(city);
   cityBox.setBounds(180, 250, 150, 30);

   l6.setBounds(50, 300, 120, 30);
   c1.setBounds(180, 300, 150, 30);

   b1.setBounds(150, 370, 100, 30);

   add(l1);
   add(t1);

   add(l2);
   add(t2);

   add(l3);
   add(t3);

   add(l4);
   add(r1);
   add(r2);

   add(l5);
   add(cityBox);

   add(l6);
   add(c1);

   add(b1);

   b1.addActionListener(this);

   setSize(450, 500);
   setVisible(true);
   setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
}

public void actionPerformed(ActionEvent e)
{
   JOptionPane.showMessageDialog(this,
         "Admission Enquiry Submitted Successfully");
}

public static void main(String args[])
{
   new Admission();
}}
```

Slip 04

Develop a Java application to manage student records using object-oriented principles. The system should support student registration, updating details, calculating grades, and displaying student information.

```
package Slip4_1;
import java.util.Scanner;
class Student
{
   int rollNo;
   String name;
   int marks1, marks2, marks3;
   double percentage;
   char grade;
   void registerStudent(){
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter Roll Number: ");
      rollNo = sc.nextInt();
      sc.nextLine();
      System.out.print("Enter Name: ");
      name = sc.nextLine();

      System.out.print("Enter Marks 1: ");
      marks1 = sc.nextInt();
      System.out.print("Enter Marks 2: ");
      marks2 = sc.nextInt();
      System.out.print("Enter Marks 3: ");
      marks3 = sc.nextInt();
   }
   void updateDetails() {
      Scanner sc = new Scanner(System.in);
```

```
        System.out.print("Enter New Name: ");
        name = sc.nextLine();
    }
    void calculateGrade(){
        int total = marks1 + marks2 + marks3;
        percentage = total / 3.0;
        if(percentage >= 75)
        {
            grade = 'A';
        }
        else if(percentage >= 60)
        {
            grade = 'B';
        }
        else if(percentage >= 40)
        {
            grade = 'C';
        }
        else
        {
            grade = 'F';
        }}
    void displayStudent()
    {
        System.out.println("\n===== Student Information =====");
        System.out.println("Roll Number : " + rollNo);
        System.out.println("Name : " + name);
        System.out.println("Percentage : " + percentage);
        System.out.println("Grade : " + grade);
    }}
public class StudentManagement{
```

```
  public static void main(String args[]){
    Student s = new Student();
    s.registerStudent();
    s.updateDetails();
    s.calculateGrade();
    s.displayStudent();
  }}
```

OR

Develop a Java application that connects to a MySQL database to manage student information. Implement CRUD operations for student records using JDBC. Use prepared statements to handle SQL queries securely and ensure proper transaction management.

```
public package Slip1_2;
import java.sql.*;
import java.util.Scanner;
public class StudentRecord {
  static final String URL = "jdbc:mysql://localhost:3306/slips";
  static final String USER = "root";
  static final String PASSWORD = "Shubham.k";

  public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);
    try {
      Connection con = DriverManager.getConnection(URL, USER, PASSWORD);
      con.setAutoCommit(false);
      int choice;
      do {
        System.out.println("\n===== STUDENT CRUD OPERATIONS =====");
        System.out.println("1. Insert Student");
```

```
System.out.println("2. Display Students");
System.out.println("3. Update Student");
System.out.println("4. Delete Student");
System.out.println("5. Exit");
System.out.print("Enter Choice: ");
choice = sc.nextInt();
switch (choice) {
   case 1:
      System.out.print("Enter Student ID: ");
      int id = sc.nextInt();
      sc.nextLine();

      System.out.print("Enter Student Name: ");
      String name = sc.nextLine();
      System.out.print("Enter Course: ");
      String course = sc.nextLine();
      String insertQuery ="INSERT INTO slip1_2 (id,name,course) VALUES(?,?,?)";

      PreparedStatement ps1 =con.prepareStatement(insertQuery);
      ps1.setInt(1, id);
      ps1.setString(2, name);
      ps1.setString(3, course);
      ps1.executeUpdate();

      con.commit();
      System.out.println("Student Inserted Successfully.");
      break;
   case 2: String selectQuery = "SELECT * FROM slip1_2";
      PreparedStatement ps2 =con.prepareStatement(selectQuery);
      ResultSet rs = ps2.executeQuery();
```

```
    System.out.println("\n--- Student Records ---");
    while (rs.next()) {
      System.out.println(
            rs.getInt("id") + "  " +
                  rs.getString("name") + "  " +
                  rs.getString("course")
      );}
    break;
  case 3:System.out.print("Enter Student ID to Update: ");
    int updateId = sc.nextInt();
    sc.nextLine();
    System.out.print("Enter New Name: ");
    String newName = sc.nextLine();
    String updateQuery ="UPDATE slip1_2 SET name=? WHERE id=?";

    PreparedStatement ps3 =con.prepareStatement(updateQuery);
    ps3.setString(1, newName);
    ps3.setInt(2, updateId);
    ps3.executeUpdate();
    con.commit();
    System.out.println("Student Updated Successfully.");
    break;
  case 4:
    System.out.print("Enter Student ID to Delete: ");
    int deleteId = sc.nextInt();
    String deleteQuery ="DELETE FROM slip1_2 WHERE id=?";

    PreparedStatement ps4 =con.prepareStatement(deleteQuery);
    ps4.setInt(1, deleteId);
    ps4.executeUpdate();
    con.commit();
```

```
                System.out.println("Student Deleted Successfully.");
                break;
            case 5: System.out.println("Program Ended.");
                break;
            default:
                System.out.println("Invalid Choice.");
          }
        } while (choice != 5);
      } catch (Exception e) {
        System.out.println(e);
      } }}
```

Slip 05

Write a program to accept roll no. Marks from users. Throw user defined exception Marks out of Bound if marks are 100.

```
import java.util.Scanner;
class MarksOutOfBoundException extends Exception{
  MarksOutOfBoundException(String msg){
    super(msg);
  }}
public class  Student   {
  public static void main(String args[]){
    Scanner sc = new Scanner(System.in);
    try {
      System.out.print("Enter Roll Number: ");
      int rollNo = sc.nextInt();
      sc.nextLine();
      System.out.print("Enter Marks: ");
      int marks = sc.nextInt();
      sc.nextLine();
      if(marks > 100){
```

```
            throw new MarksOutOfBoundException(
                  "Marks Out Of Bound");
         }
         System.out.println("Roll No: " + rollNo);
         System.out.println("Marks: " + marks);
      }
      catch(MarksOutOfBoundException e){
         System.out.println(e);
      }
      sc.close();
   }}
```

OR

Implement a Java program to manage a library system using OOP concepts such as classes, inheritance, encapsulation, and polymorphism. Include features like adding books, issuing books, returning books, and checking availability.

```
import java.util.Scanner;
class Book{
   int id;
   String name;
   boolean available;
   Book(int id, String name){
      this.id = id;
      this.name = name;
      available = true;
   }}
public class Books
{
   public static void main(String args[]) {
      Scanner sc = new Scanner(System.in);
      Book books[] = new Book[10];
      int count = 0;
```

```
int choice;
do  {
   System.out.println("\n===== LIBRARY MENU =====");
   System.out.println("1. Add Book");
   System.out.println("2. Issue Book");
   System.out.println("3. Return Book");
   System.out.println("4. Display Books");
   System.out.println("5. Exit");

   System.out.print("Enter Choice: ");
   choice = sc.nextInt();
   switch(choice) {
      case 1:
         System.out.print("Enter Book ID: ");
         int id = sc.nextInt();
         sc.nextLine();
         System.out.print("Enter Book Name: ");
         String name = sc.nextLine();
         books[count] = new Book(id, name);
         count++;
         System.out.println("Book Added Successfully");
         break;
      case 2:
         System.out.print("Enter Book ID to Issue: ");
         int issueId = sc.nextInt();
         for(int i = 0; i < count; i++){
            if(books[i].id == issueId){
               if(books[i].available){
                  books[i].available = false;
                  System.out.println("Book Issued");
               }
```

```
            else  {
                System.out.println("Book Already Issued");
            }}}
        break;
    case 3:
        System.out.print("Enter Book ID to Return: ");
        int returnId = sc.nextInt();
        for(int i = 0; i < count; i++){
            if(books[i].id == returnId){
                books[i].available = true;
                System.out.println("Book Returned");
            } }
        break;
    case 4:
        System.out.println("\n===== BOOK DETAILS =====");
        for(int i = 0; i < count; i++){
            System.out.println("Book ID : " + books[i].id);
            System.out.println("Book Name : " + books[i].name);
            if(books[i].available){
                System.out.println("Status : Available");
            }
            else{
                System.out.println("Status : Issued");
            }
            System.out.println("-------------------");
        }
        break;
    case 5:
        System.out.println("Program Ended");
        break;
    default:
```

```
                System.out.println("Invalid Choice");
            }
        } while(choice != 5);
        sc.close();
    }}
```

Slip 06

Create a Java program to model an academic institution using OOP concepts like inheritance, encapsulation, and polymorphism. Define a class hierarchy as follows: Staff → Professor → Head of Department (HOD).

A. Staff: Store and display staff name, date of joining, and staff ID.

B. Professor: Inherit all Staff details and additionally display subject specialization and salary.

C. HOD: Inherit all Professor details and additionally display department allowance (if applicable).

```
package Slip6_1;
import java.util.Scanner;

class Staff
{
    int staffId;
    String name;
    String doj;

    Staff(int staffId, String name, String doj)
    {
        this.staffId = staffId;
        this.name = name;
        this.doj = doj;
    }

    void displayStaff()
    {
        System.out.println("\n===== STAFF DETAILS =====");
        System.out.println("Staff ID : " + staffId);
        System.out.println("Staff Name : " + name);
        System.out.println("Date of Joining : " + doj);
    }
}
```

```
class Professor extends Staff
{
   String subject;
   double salary;

   Professor(int staffId, String name, String doj,
         String subject, double salary)
   {
      super(staffId, name, doj);

      this.subject = subject;
      this.salary = salary;
   }

   void displayProfessor()
   {
      System.out.println("\n===== PROFESSOR DETAILS =====");

      System.out.println("Subject : " + subject);
      System.out.println("Salary : " + salary);
   }
}

class HOD extends Professor
{
   double allowance;

   HOD(int staffId, String name, String doj,
      String subject, double salary, double allowance)
   {
      super(staffId, name, doj, subject, salary);

      this.allowance = allowance;
   }

   void displayHOD()
   {
      System.out.println("\n===== HOD DETAILS =====");

      System.out.println("Department Allowance : " + allowance);
   }
}

public class Institution
{
   public static void main(String args[])
```

```
{
    Scanner sc = new Scanner(System.in);

    System.out.print("Enter Staff ID: ");
    int id = sc.nextInt();
    sc.nextLine();

    System.out.print("Enter Name: ");
    String name = sc.nextLine();

    System.out.print("Enter Date of Joining: ");
    String doj = sc.nextLine();

    System.out.print("Enter Subject: ");
    String subject = sc.nextLine();

    System.out.print("Enter Salary: ");
    double salary = sc.nextDouble();

    System.out.print("Enter Department Allowance: ");
    double allowance = sc.nextDouble();

    Staff s = new Staff(id, name, doj);

    Professor p = new Professor(id, name, doj,subject, salary);

    HOD h = new HOD(id, name, doj, subject, salary, allowance);

    s.displayStaff();

    p.displayProfessor();

    h.displayHOD();

    sc.close();
}}
```

OR

Write a program of servlet to demonstrate GET and POST Methods with suitable example using simulation.

```
package Slip3_2;

/*Write a program to design an admission enquiry for MCA student form using
Swing.*/
```

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class Admission extends JFrame implements ActionListener
{
   JLabel l1, l2, l3, l4, l5, l6;
   JTextField t1, t2, t3;
   JRadioButton r1, r2;
   JComboBox c1;
   JButton b1;

   Admission()
   {
      setTitle("MCA Admission Enquiry Form");

      l1 = new JLabel("Student Name:");
      l2 = new JLabel("Mobile Number:");
      l3 = new JLabel("Email ID:");
      l4 = new JLabel("Gender:");
      l5 = new JLabel("City:");
      l6 = new JLabel("Course:");

      t1 = new JTextField();
      t2 = new JTextField();
      t3 = new JTextField();

      r1 = new JRadioButton("Male");
      r2 = new JRadioButton("Female");

      ButtonGroup bg = new ButtonGroup();
      bg.add(r1);
      bg.add(r2);

      String course[] = {"MCA", "MBA", "MSc IT"};
      c1 = new JComboBox(course);

      b1 = new JButton("Submit");

      setLayout(null);

      l1.setBounds(50, 50, 120, 30);
      t1.setBounds(180, 50, 150, 30);

      l2.setBounds(50, 100, 120, 30);
```

```
    t2.setBounds(180, 100, 150, 30);

    l3.setBounds(50, 150, 120, 30);
    t3.setBounds(180, 150, 150, 30);

    l4.setBounds(50, 200, 120, 30);
    r1.setBounds(180, 200, 80, 30);
    r2.setBounds(260, 200, 80, 30);

    l5.setBounds(50, 250, 120, 30);

    String city[] = {"Mumbai", "Pune", "Nashik"};
    JComboBox cityBox = new JComboBox(city);
    cityBox.setBounds(180, 250, 150, 30);

    l6.setBounds(50, 300, 120, 30);
    c1.setBounds(180, 300, 150, 30);

    b1.setBounds(150, 370, 100, 30);

    add(l1);
    add(t1);

    add(l2);
    add(t2);

    add(l3);
    add(t3);

    add(l4);
    add(r1);
    add(r2);

    add(l5);
    add(cityBox);

    add(l6);
    add(c1);

    add(b1);

    b1.addActionListener(this);

    setSize(450, 500);
    setVisible(true);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
}
```

```
    public void actionPerformed(ActionEvent e)
    {
        JOptionPane.showMessageDialog(this,
                "Admission Enquiry Submitted Successfully");
    }

    public static void main(String args[])
    {
        new Admission();
    }
}
```

Slip 07

Implement a Java Program for multithreading (Synchronization): Create thread for Withdraw & Deposit tasks for Bank Account, display the current balance also.

```
class BankAccount {
    private int balance = 1000; // Initial Balance

    // Synchronized method for deposit
    public synchronized void deposit(int amount) {
        System.out.println("Depositing Amount: " + amount);

        balance = balance + amount;

        System.out.println("Amount Deposited Successfully");
        System.out.println("Current Balance after Deposit: " + balance);
        System.out.println("--------------------------------");
    }

    // Synchronized method for withdraw
    public synchronized void withdraw(int amount) {
        System.out.println("Withdrawing Amount: " + amount);

        if (balance >= amount) {
            balance = balance - amount;

            System.out.println("Amount Withdrawn Successfully");
            System.out.println("Current Balance after Withdrawal: " + balance);
        } else {
            System.out.println("Insufficient Balance");
            System.out.println("Current Balance: " + balance);
```

```
        }

        System.out.println("--------------------------------");
    }

    // Method to display current balance
    public synchronized void displayBalance() {
        System.out.println("Final Current Balance: " + balance);
    }
}

// Deposit Thread
class DepositThread extends Thread {
    BankAccount account;

    DepositThread(BankAccount account) {
        this.account = account;
    }

    public void run() {
        account.deposit(500);
    }
}
// Withdraw Thread
class WithdrawThread extends Thread {
    BankAccount account;

    WithdrawThread(BankAccount account) {
        this.account = account;
    }

    public void run() {
        account.withdraw(700);
    }
}

// Main Class
public class BankSynchronization {
    public static void main(String[] args) {

        BankAccount account = new BankAccount();

        // Creating Threads
        DepositThread t1 = new DepositThread(account);
        WithdrawThread t2 = new WithdrawThread(account);

        // Starting Threads
```

```
        t1.start();
        t2.start();

        // Waiting for threads to complete
        try {
            t1.join();
            t2.join();
        } catch (InterruptedException e) {
            System.out.println(e);
        }

        // Display Final Balance
        account.displayBalance();
    }
}
```

OR

Write a program to demonstrate how the process tasks asynchronously in a Spring Boot application.

```
/* 1.go to start.spring.io
2.download file
3.open the file in intellij and run the application
4.go to src/main/java/com/example/demo/DemoApplication.java
5.create class AsyncService,AsyncController,AsyncDemoApplication */


/* ADD code in AsyncDemoApplication.java */
package com.example.firstdemo;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.scheduling.annotation.EnableAsync;
@SpringBootApplication
@EnableAsync
public class AsyncDemoApplication {
    public static void main (String[] args) {
        SpringApplication.run(AsyncDemoApplication.class, args);
    }}
```

```
/* Add code in AsyncService.java */
package com.example.firstdemo;
import org.springframework.scheduling.annotation.Async;
import org.springframework.stereotype.Service;
@Service
public class AsyncService {
   @Async
   public void processTask() {
      try {
         System.out.println("Task started on thread: " +
               Thread.currentThread().getName());
         Thread.sleep(5000); // simulate long process (5 seconds)
         System.out.println("Task completed on thread:" +
               Thread.currentThread().getName());
      } catch (InterruptedException e) {
         e.printStackTrace();
      } }}

/* Add code in AsyncController.java */
package com.example.firstdemo;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
@RestController
public class AsyncController {
   @Autowired
   private AsyncService asyncService;
   @GetMapping("/run")
   public String runTask() {
      asyncService.processTask();
      return "Task triggered successfully";
```

```
}}
```

Slip 08

Implement a Java program to manage bank accounts using object-oriented programming (OOP) concepts like classes, inheritance, encapsulation, and polymorphism. Include features like account creation, deposit, withdrawal, and balance inquiry.

```
package Slip8_1;
import java.util.Scanner;
class Account
{
   int accNo;
   String name;
   double balance;
   void createAccount(int no, String n, double bal)
   {
      accNo = no;
      name = n;
      balance = bal;
      System.out.println("Account Created Successfully");
   }
   void deposit(double amount)
   {
      balance = balance + amount;
      System.out.println("Amount Deposited");
   }
   void withdraw(double amount){
      if(balance >= amount) {
         balance = balance - amount;
         System.out.println("Amount Withdrawn");
      }
```

```
        else{
            System.out.println("Insufficient Balance");
        }}
    void checkBalance()
    {
        System.out.println("\nAccount No : " + accNo);
        System.out.println("Name : " + name);
        System.out.println("Balance : " + balance);
    }}
class SavingsAccount extends Account{
    void checkBalance()   // Polymorphism {
        System.out.println("\nSavings Account Balance : " + balance);
    }}
public class BankSystem
{
    public static void main(String args[])
    {
        Scanner sc = new Scanner(System.in);
        SavingsAccount a = new SavingsAccount();
        int choice;
        do
        {
            System.out.println("\n===== BANK MENU =====");
            System.out.println("1. Create Account");
            System.out.println("2. Deposit");
            System.out.println("3. Withdraw");
            System.out.println("4. Check Balance");
            System.out.println("5. Exit");
            System.out.print("Enter Choice: ");
            choice = sc.nextInt();
```

```
switch(choice)
{
   case 1:
      System.out.print("Enter Account Number: ");
      int no = sc.nextInt();
      sc.nextLine();
      System.out.print("Enter Name: ");
      String name = sc.nextLine();
      System.out.print("Enter Initial Balance: ");
      double bal = sc.nextDouble();
      a.createAccount(no, name, bal);
      break;
   case 2:
      System.out.print("Enter Deposit Amount: ");
      double dep = sc.nextDouble();
      a.deposit(dep);
      break;
   case 3:
      System.out.print("Enter Withdraw Amount: ");
      double wd = sc.nextDouble();
      a.withdraw(wd);
      break;

   case 4:
      a.checkBalance();
      break;
   case 5:
      System.out.println("Program Ended");
      break;
   default:
      System.out.println("Invalid Choice");
```

```
        }
    } while(choice != 5);
    sc.close();
}}
```

OR

Write a program to demonstrate how the process tasks asynchronously in a Spring Boot application.

```
/* 1.go to start.spring.io
2.download file
3.open the file in intellij and run the application
4.go to src/main/java/com/example/demo/DemoApplication.java
5.create class AsyncService,AsyncController,AsyncDemoApplication */

/* ADD code in AsyncDemoApplication.java */
package com.example.firstdemo;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.scheduling.annotation.EnableAsync;
@SpringBootApplication
@EnableAsync
public class AsyncDemoApplication {
    public static void main (String[] args) {
        SpringApplication.run(AsyncDemoApplication.class, args);
    }}

/* Add code in AsyncService.java */
package com.example.firstdemo;
import org.springframework.scheduling.annotation.Async;
import org.springframework.stereotype.Service;
@Service
public class AsyncService {
```

```
    @Async
    public void processTask() {
        try {
            System.out.println("Task started on thread: " +
                    Thread.currentThread().getName());
            Thread.sleep(5000); // simulate long process (5 seconds)
            System.out.println("Task completed on thread:" +
                    Thread.currentThread().getName());
        } catch (InterruptedException e) {
            e.printStackTrace();
        } }}
/* Add code in AsyncController.java */
package com.example.firstdemo;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
@RestController
public class AsyncController {
    @Autowired
    private AsyncService asyncService;
    @GetMapping("/run")
    public String runTask() {
        asyncService.processTask();
        return "Task triggered successfully";
    }}
```

Slip 09

Develop a Java-based library management application that demonstrates OOP principles and uses exception handling to handle cases like issuing books, returning books, etc.

```
import java.util.Scanner;
class BookException extends Exception
{
   BookException(String msg)
   {
      super(msg);
   }
}
class Book
{
   int id;
   String name;
   boolean available = true;

   void addBook(int i, String n)
   {
      id = i;
      name = n;
      System.out.println("Book Added Successfully");
   }

   void issueBook() throws BookException
   {
      if(available)
      {
         available = false;
         System.out.println("Book Issued Successfully");
      }
      else
      {
         throw new BookException("Book Already Issued");
      }}
   void returnBook() throws BookException
   {
      if(!available)
      {
         available = true;
         System.out.println("Book Returned Successfully");
      }

      else
      {
         throw new BookException("Book Already Available");
      }
   }
   void displayBook()
   {
```

```
        System.out.println("\nBook ID : " + id);
        System.out.println("Book Name : " + name);
       if(available)
        {
           System.out.println("Status : Available");
        }

        else
        {
           System.out.println("Status : Issued");
        }}}
public class LibraryManagement
{
   public static void main(String args[])
   {
      Scanner sc = new Scanner(System.in);
      Book b = new Book();
      int choice;

      do
      {
         System.out.println("\n===== LIBRARY MENU =====");
         System.out.println("1. Add Book");
         System.out.println("2. Issue Book");
         System.out.println("3. Return Book");
         System.out.println("4. Display Book");
         System.out.println("5. Exit");

         System.out.print("Enter Choice: ");
         choice = sc.nextInt();

         try
         {
            switch(choice)
            {
               case 1:
                  System.out.print("Enter Book ID: ");
                  int id = sc.nextInt();
                  sc.nextLine();

                  System.out.print("Enter Book Name: ");
                  String name = sc.nextLine();

                  b.addBook(id, name);
                  break;
               case 2:
                  b.issueBook();
```

```
                break;
            case 3:
                b.returnBook();
                break;
            case 4:
                b.displayBook();
                break;
            case 5:
                System.out.println("Program Ended");
                break;
            default:
                System.out.println("Invalid Choice");
        }
      }
      catch(BookException e)
      {
        System.out.println(e.getMessage());
      }
    } while(choice != 5);
    sc.close();
  }}
```

OR

Write a client-server program using TCP sockets to echo the message sent by the client.

Server.java

```
import java.io.*;
import java.net.*;
public class Server
{
   public static void main(String args[]) throws Exception{
      ServerSocket ss = new ServerSocket(5000);
      System.out.println("Server Waiting...");
      Socket s = ss.accept();
      DataInputStream dis =new DataInputStream(s.getInputStream());
      String msg = dis.readUTF();
      System.out.println("Client Message : " + msg);

      DataOutputStream dos =new DataOutputStream(s.getOutputStream());
```

```
        dos.writeUTF("Echo : " + msg);
        ss.close();
    }}
```

Client.java

```
import java.io.*;
import java.net.*;
import java.util.Scanner;
public class Client
{
    public static void main(String args[]) throws Exception{
        Socket s = new Socket("localhost", 5000);
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Message : ");
        String msg = sc.nextLine();
        DataOutputStream dos =new DataOutputStream(s.getOutputStream());
        dos.writeUTF(msg);
        DataInputStream dis =new DataInputStream(s.getInputStream());
        String serverMsg = dis.readUTF();
        System.out.println("Server Reply : " + serverMsg);
        s.close();
    }}
```

Slip 10

Implement a Java program to manage retail staff using OOP concepts. Define a hierarchy: Worker → Supervisor → StoreManager.

A. Worker: Store and display worker name, ID, and date of birth.

B. Supervisor: Inherit Worker details and additionally include monthly salary.

C. StoreManager: Inherit Supervisor details and additionally include

profit-based incentives or commission (if applicable).

```
class Worker
{
   int id;
   String name;
   String dob;

   Worker(int id, String name, String dob)
   {
      this.id = id;
      this.name = name;
      this.dob = dob;
   }
   void displayWorker()
   {
      System.out.println("\n===== WORKER DETAILS =====");
      System.out.println("Worker ID : " + id);
      System.out.println("Worker Name : " + name);
      System.out.println("Date of Birth : " + dob);
   }
}

class Supervisor extends Worker
{
   double salary;

   Supervisor(int id, String name,
            String dob, double salary)
   {
      super(id, name, dob);

      this.salary = salary;
   }
   void displaySupervisor()
   {
      System.out.println("\n===== SUPERVISOR DETAILS =====");

      System.out.println("Monthly Salary : " + salary);
   }}

class StoreManager extends Supervisor
{
   double incentive;

   StoreManager(int id, String name,
              String dob, double salary,
```

```
            double incentive)
    {
        super(id, name, dob, salary);

        this.incentive = incentive;
    }
    void displayManager()
    {
        System.out.println("\n===== STORE MANAGER DETAILS =====");

        System.out.println("Profit Incentive : " + incentive);
    }}
public class RetailStaff
{
    public static void main(String args[])
    {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Worker ID: ");
        int id = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter Worker Name: ");
        String name = sc.nextLine();

        System.out.print("Enter Date of Birth: ");
        String dob = sc.nextLine();

        System.out.print("Enter Monthly Salary: ");
        double salary = sc.nextDouble();

        System.out.print("Enter Incentive: ");
        double incentive = sc.nextDouble();

        Worker w =new Worker(id, name, dob);
        Supervisor s = new Supervisor(id, name,dob, salary);

        StoreManager sm =new StoreManager(id, name, dob, salary, incentive);
       w.displayWorker();
        s.displaySupervisor();

        sm.displayManager();
        sc.close();
    }}
```

OR

Write a client-server program using TCP sockets to echo the message sent by the

client.

Server.java

```
package Slip9_2;
import java.io.*;
import java.net.*;
public class Server
{
   public static void main(String args[]) throws Exception{
      ServerSocket ss = new ServerSocket(5000);
      System.out.println("Server Waiting...");
      Socket s = ss.accept();
      DataInputStream dis =new DataInputStream(s.getInputStream());
      String msg = dis.readUTF();
      System.out.println("Client Message : " + msg);

      DataOutputStream dos =new DataOutputStream(s.getOutputStream());
      dos.writeUTF("Echo : " + msg);
      ss.close();
   }}
```

Client.java

```
package Slip9_2;
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class Client
{
   public static void main(String args[]) throws Exception{
      Socket s = new Socket("localhost", 5000);
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter Message : ");
```

```
    String msg = sc.nextLine();
    DataOutputStream dos =new DataOutputStream(s.getOutputStream());
    dos.writeUTF(msg);
    DataInputStream dis =new DataInputStream(s.getInputStream());
    String serverMsg = dis.readUTF();
    System.out.println("Server Reply : " + serverMsg);
    s.close();
  }}
```

Slip 11

Implement a Java Program to create calculator using AWT

```
import java.awt.*;
import java.awt.event.*;

public class Calculator extends Frame implements ActionListener
{
  TextField t;

  Button b1,b2,b3,b4,b5,b6,b7,b8,b9,b0;
  Button add,sub,mul,div,eq,clear;

  double num1, num2, result;
  String op;

  Calculator()
  {
    t = new TextField();

    b1 = new Button("1");
    b2 = new Button("2");
    b3 = new Button("3");
    b4 = new Button("4");
    b5 = new Button("5");
    b6 = new Button("6");
    b7 = new Button("7");
    b8 = new Button("8");
    b9 = new Button("9");
    b0 = new Button("0");

    add = new Button("+");
```

```
    sub = new Button("-");
    mul = new Button("*");
    div = new Button("/");
    eq = new Button("=");

    clear = new Button("C");

    setLayout(new GridLayout(5,4));

    add(t);

    add(b1); add(b2); add(b3);
    add(b4); add(b5); add(b6);
    add(b7); add(b8); add(b9);

    add(b0);

    add(add);
    add(sub);
    add(mul);
    add(div);

    add(eq);
    add(clear);

    b1.addActionListener(this);
    b2.addActionListener(this);
    b3.addActionListener(this);
    b4.addActionListener(this);
    b5.addActionListener(this);
    b6.addActionListener(this);
    b7.addActionListener(this);
    b8.addActionListener(this);
    b9.addActionListener(this);
    b0.addActionListener(this);

    add.addActionListener(this);
    sub.addActionListener(this);
    mul.addActionListener(this);
    div.addActionListener(this);
    eq.addActionListener(this);

    clear.addActionListener(this);

    setSize(300,300);
    setVisible(true);
}
```

```
public void actionPerformed(ActionEvent e)
{
   String s = e.getActionCommand();

   if(s.equals("1") || s.equals("2") ||
         s.equals("3") || s.equals("4") ||
         s.equals("5") || s.equals("6") ||
         s.equals("7") || s.equals("8") ||
         s.equals("9") || s.equals("0"))
   {
      t.setText(t.getText() + s);
   }

   else if(s.equals("+") || s.equals("-") ||
         s.equals("*") || s.equals("/"))
   {
      num1 = Double.parseDouble(t.getText());
      op = s;
      t.setText("");
   }

   else if(s.equals("="))
   {
      num2 = Double.parseDouble(t.getText());

      if(op.equals("+"))
      {
         result = num1 + num2;
      }

      else if(op.equals("-"))
      {
         result = num1 - num2;
      }

      else if(op.equals("*"))
      {
         result = num1 * num2;
      }

      else if(op.equals("/"))
      {
         result = num1 / num2;
      }

      t.setText("" + result);
```

```
    }

    else if(s.equals("C"))
    {
      t.setText("");
    }
  }

  public static void main(String args[])
  {
    new Calculator();
  }}
```

OR

Write a program to implement student information in a file and perform the file operations on it.

```
import java.io.*;
import java.util.*;
public class StudentFile
{
  public static void main(String args[]) throws Exception
  {
    Scanner sc = new Scanner(System.in);
    int choice;
    do{
      System.out.println("\n1.Add Student");
      System.out.println("2.View Students");
      System.out.println("3.Delete by Roll No");
      System.out.println("4.Exit");
      System.out.print("Enter choice: ");
      choice = sc.nextInt();
      switch(choice)
      {
        case 1: FileWriter fw =new FileWriter("student.txt", true);
          sc.nextLine();
          System.out.print("Enter Roll No: ");
```

```
    String roll = sc.nextLine();
    System.out.print("Enter Name: ");
    String name = sc.nextLine();

    System.out.print("Enter Course: ");
    String course = sc.nextLine();

    fw.write(roll + "," + name + "," +course + "\n");
    fw.close();
    System.out.println("Student Added");
    break;
case 2:
    BufferedReader br =new BufferedReader(new FileReader("student.txt"));
    String line;
    System.out.println("\nStudent Records:");
    while((line = br.readLine()) != null)
    {
       System.out.println(line);
    }
    br.close();
    break;
case 3:sc.nextLine();
    System.out.print("Enter Roll No to Delete: ");
    String deleteRoll =sc.nextLine();
    File input =new File("student.txt");
    File temp =new File("temp.txt");
    BufferedReader reader =new BufferedReader(new FileReader(input));
    BufferedWriter writer =new BufferedWriter(new FileWriter(temp));
    String currentLine;
    while((currentLine =reader.readLine()) != null)
    {
```

```
                String data[] =currentLine.split(",");
                if(!data[0].equals(deleteRoll))
                {
                   writer.write(
                         currentLine + "\n");
                }}
             reader.close();
             writer.close();
             input.delete();
             temp.renameTo(input);
             System.out.println("Student Deleted");
             break;
          case 4:System.out.println("Program Ended");
             break;
          default: System.out.println("Invalid Choice");
        }
     } while(choice != 4);
     sc.close();
  }}
```

Slip 12

Write a program to accept roll no. Marks from users. Throw user defined exception Marks out of Bound if marks are 100.

```
package Slip12_1;
import java.util.Scanner;
class MarksOutOfBoundException extends Exception
{
   MarksOutOfBoundException(String msg)
   {
      super(msg);
   }}
public class  Student {
```

```
  public static void main(String args[]){
    Scanner sc = new Scanner(System.in);
    try
    {
      System.out.print("Enter Roll Number: ");
      int rollNo = sc.nextInt();
      sc.nextLine();
      System.out.print("Enter Marks: ");
      int marks = sc.nextInt();
      sc.nextLine();
      if(marks > 100)
      {
        throw new MarksOutOfBoundException(
            "Marks Out Of Bound");
      }
      System.out.println("Roll No: " + rollNo);
      System.out.println("Marks: " + marks);
    }
    catch(MarksOutOfBoundException e){
      System.out.println(e);
    }
    sc.close();
  }}
```

OR

Create a Java application that connects to a MySQL database to manage employee information. Implement functionalities like adding, updating, deleting, and viewing employee records using JDBC. Use prepared statements to prevent SQL injection and handle exceptions gracefully.

```
import java.sql.*;
import java.util.Scanner;
```

```
public class EmployeeCRUD
{
   static final String URL ="jdbc:mysql://localhost:3306/slips";
   static final String USER = "root";
   static final String PASSWORD = "Shubham.k";
   public static void main(String args[])
   {
      Scanner sc = new Scanner(System.in);
      try
      {
         Connection con =DriverManager.getConnection(URL, USER, PASSWORD);
         int choice;

         do
         {
            System.out.println("\n===== EMPLOYEE MENU =====");
            System.out.println("1. Add Employee");
            System.out.println("2. Update Employee");
            System.out.println("3. Delete Employee");
            System.out.println("4. View Employees");
            System.out.println("5. Exit");

            System.out.print("Enter Choice: ");
            choice = sc.nextInt();
            switch(choice)
            {
               case 1:
                  System.out.print("Enter Employee ID: ");
                  int id = sc.nextInt();
                  sc.nextLine();

                  System.out.print("Enter Employee Name: ");
                  String name = sc.nextLine();

                  System.out.print("Enter Salary: ");
                  double salary = sc.nextDouble();

                  String insertQuery = "INSERT INTO slip19_2 VALUES(?,?,?)";
                  PreparedStatement ps1 =con.prepareStatement(insertQuery);

                  ps1.setInt(1, id);
                  ps1.setString(2, name);
                  ps1.setDouble(3, salary);
                  ps1.executeUpdate();

                  System.out.println("Employee Added Successfully");
```

```
    break;
// Update Employee
case 2:
    System.out.print("Enter Employee ID: ");
    int updateId = sc.nextInt();
    sc.nextLine();

    System.out.print("Enter New Name: ");
    String newName = sc.nextLine();

    String updateQuery = "UPDATE slip19_2 SET name=? WHERE id=?";
    PreparedStatement ps2 =con.prepareStatement(updateQuery);

    ps2.setString(1, newName);
    ps2.setInt(2, updateId);

    ps2.executeUpdate();
    System.out.println("Employee Updated Successfully");
    break;

// Delete Employee
case 3:
    System.out.print("Enter Employee ID: ");
    int deleteId = sc.nextInt();
    String deleteQuery = "DELETE FROM slip19_2 WHERE id=?";
    PreparedStatement ps3 =con.prepareStatement(deleteQuery);

    ps3.setInt(1, deleteId);

    ps3.executeUpdate();
    System.out.println("Employee Deleted Successfully");
    break;
// View Employees
case 4:
    String selectQuery ="SELECT * FROM slip19_2";

    PreparedStatement ps4 = con.prepareStatement(selectQuery);

    ResultSet rs = ps4.executeQuery();

    System.out.println("\n===== EMPLOYEE RECORDS =====");

    while(rs.next())
    {
        System.out.println(
            rs.getInt("id") + "  " +
                rs.getString("name") + "  " +
```

```
                        rs.getDouble("salary"));
                }
                break;
            case 5:
                System.out.println("Program Ended");
                break;
            default:
                System.out.println("Invalid Choice");
        }

      } while(choice != 5);

      con.close();
    }
    catch(Exception e)
    {
      System.out.println(e);
    }
    sc.close();
  }}
```

Slip 13

Develop a Java-based library management application that demonstrates OOP principles and uses exception handling to handle cases like issuing books, returning books, etc.

```
import java.util.Scanner;
class BookException extends Exception
{
  BookException(String msg)
  {
    super(msg);
  }}
class Book
{
  int id;
  String name;
  boolean available = true;
  void addBook(int i, String n)
```

```
{
   id = i;
   name = n;
   System.out.println("Book Added Successfully");
}
void issueBook() throws BookException
{
   if(available)
   {
      available = false;
      System.out.println("Book Issued Successfully");
   }
   else {
      throw new BookException("Book Already Issued");
   }}
void returnBook() throws BookException
{
   if(!available)   {
      available = true;
      System.out.println("Book Returned Successfully");
   }
   else
   {
      throw new BookException("Book Already Available");
   }}
void displayBook()
{
   System.out.println("\nBook ID : " + id);
   System.out.println("Book Name : " + name);
   if(available)
   {
```

```
            System.out.println("Status : Available");
        }
        else  {
            System.out.println("Status : Issued");
        }
    }}
public class LibraryManagement
{
    public static void main(String args[]){
        Scanner sc = new Scanner(System.in);
        Book b = new Book();
        int choice;
        do {
            System.out.println("\n===== LIBRARY MENU =====");
            System.out.println("1. Add Book");
            System.out.println("2. Issue Book");
            System.out.println("3. Return Book");
            System.out.println("4. Display Book");
            System.out.println("5. Exit");
            System.out.print("Enter Choice: ");
            choice = sc.nextInt();
            try{
                switch(choice){
                    case 1:
                        System.out.print("Enter Book ID: ");
                        int id = sc.nextInt();
                        sc.nextLine();
                        System.out.print("Enter Book Name: ");
                        String name = sc.nextLine();
             b.addBook(id, name);
                        break;
```

```
                case 2:b.issueBook();
                   break;
                case 3:b.returnBook();
                   break;
                case 4:b.displayBook();
                   break;
                case 5:System.out.println("Program Ended");
                   break;
                default:
                   System.out.println("Invalid Choice");
             }}
          catch(BookException e){
             System.out.println(e.getMessage());
          }
       } while(choice != 5);
       sc.close();
    }}
```

OR

Write a program to design an admission enquiry form using Swing.

```
/*Write a program to design an admission enquiry for MCA student form using
Swing.*/


import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class MCAAdmissionForm extends JFrame implements ActionListener
{
   JLabel l1, l2, l3, l4, l5, l6;
   JTextField t1, t2, t3;
```

```
JRadioButton r1, r2;
JComboBox c1;
JButton b1;

MCAAdmissionForm()
{
   setTitle("MCA Admission Enquiry Form");

   l1 = new JLabel("Student Name:");
   l2 = new JLabel("Mobile Number:");
   l3 = new JLabel("Email ID:");
   l4 = new JLabel("Gender:");
   l5 = new JLabel("City:");
   l6 = new JLabel("Course:");

   t1 = new JTextField();
   t2 = new JTextField();
   t3 = new JTextField();

   r1 = new JRadioButton("Male");
   r2 = new JRadioButton("Female");

   ButtonGroup bg = new ButtonGroup();
   bg.add(r1);
   bg.add(r2);

   String course[] = {"MCA", "MBA", "MSc IT"};
   c1 = new JComboBox(course);

   b1 = new JButton("Submit");
```

```
setLayout(null);

l1.setBounds(50, 50, 120, 30);
t1.setBounds(180, 50, 150, 30);

l2.setBounds(50, 100, 120, 30);
t2.setBounds(180, 100, 150, 30);

l3.setBounds(50, 150, 120, 30);
t3.setBounds(180, 150, 150, 30);

l4.setBounds(50, 200, 120, 30);
r1.setBounds(180, 200, 80, 30);
r2.setBounds(260, 200, 80, 30);

l5.setBounds(50, 250, 120, 30);

String city[] = {"Mumbai", "Pune", "Nashik"};
JComboBox cityBox = new JComboBox(city);
cityBox.setBounds(180, 250, 150, 30);

l6.setBounds(50, 300, 120, 30);
c1.setBounds(180, 300, 150, 30);

b1.setBounds(150, 370, 100, 30);

add(l1);
add(t1);

add(l2);
add(t2);
```

```
    add(l3);
    add(t3);

    add(l4);
    add(r1);
    add(r2);

    add(l5);
    add(cityBox);

    add(l6);
    add(c1);

    add(b1);

    b1.addActionListener(this);

    setSize(450, 500);
    setVisible(true);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
}

public void actionPerformed(ActionEvent e)
{
    JOptionPane.showMessageDialog(this,
            "Admission Enquiry Submitted Successfully");
}

public static void main(String args[])
{
```

```
        new MCAAdmissionForm();
    }
}
```

Slip 14

Implement a Java Program for developing a word counter using AWT & Swing.

```
package Slip14_1;

/*Implement a Java Program for developing a word counter using AWT & Swing.*/

import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

class WordCounter extends JFrame implements ActionListener
{
    TextArea ta;

    Label l1, l2;

    Button b;

    WordCounter()
    {
        ta = new TextArea();

        l1 = new Label("Words: 0");

        l2 = new Label("Characters: 0");

        b = new Button("Count");

        setLayout(null);

        ta.setBounds(50, 50, 300, 150);

        b.setBounds(130, 220, 100, 30);

        l1.setBounds(50, 280, 120, 30);

        l2.setBounds(200, 280, 150, 30);

        add(ta);
        add(b);
```

```
        add(l1);
        add(l2);

        b.addActionListener(this);

        setTitle("Word Counter");

        setSize(420, 400);

        setVisible(true);

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }

    public void actionPerformed(ActionEvent e)
    {
        String text = ta.getText();

        int characters = text.length();

        String words[] =
              text.trim().split("\\s+");

        int wordCount = 0;

        if(text.trim().isEmpty())
        {
           wordCount = 0;
        }
        else
        {
           wordCount = words.length;
        }

        l1.setText("Words: " + wordCount);

        l2.setText(
              "Characters: " + characters);
    }

    public static void main(String args[])
    {
        new WordCounter();
    }
}
```

OR

Write a client-server program using TCP sockets to echo the message sent by the client.

Server.java

```
import java.io.*;
import java.net.*;
public class Server
{
   public static void main(String args[]) throws Exception{
      ServerSocket ss = new ServerSocket(5000);
      System.out.println("Server Waiting...");
      Socket s = ss.accept();
      DataInputStream dis =new DataInputStream(s.getInputStream());
      String msg = dis.readUTF();
      System.out.println("Client Message : " + msg);

      DataOutputStream dos =new DataOutputStream(s.getOutputStream());
      dos.writeUTF("Echo : " + msg);
      ss.close();
   }}
```

Client.java

```
import java.io.*;
import java.net.*;
import java.util.Scanner;
public class Client
{
   public static void main(String args[]) throws Exception{
      Socket s = new Socket("localhost", 5000);
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter Message : ");
```

```
    String msg = sc.nextLine();
    DataOutputStream dos =new DataOutputStream(s.getOutputStream());
    dos.writeUTF(msg);
    DataInputStream dis =new DataInputStream(s.getInputStream());
    String serverMsg = dis.readUTF();
    System.out.println("Server Reply : " + serverMsg);
    s.close();
  }}
```

Slip 15

Implement a Java Program for multithreading (Synchronization): Create thread for Withdraw & Deposit tasks for Bank Account, display the current balance also.

```
class BankAccount {
  private int balance = 1000; // Initial Balance

  // Synchronized method for deposit
  public synchronized void deposit(int amount) {
    System.out.println("Depositing Amount: " + amount);

    balance = balance + amount;

    System.out.println("Amount Deposited Successfully");
    System.out.println("Current Balance after Deposit: " + balance);
    System.out.println("--------------------------------");
  }
  // Synchronized method for withdraw
  public synchronized void withdraw(int amount) {
    System.out.println("Withdrawing Amount: " + amount);

    if (balance >= amount) {
      balance = balance - amount;

      System.out.println("Amount Withdrawn Successfully");
      System.out.println("Current Balance after Withdrawal: " + balance);
    } else {
      System.out.println("Insufficient Balance");
      System.out.println("Current Balance: " + balance);
    }

    System.out.println("--------------------------------");
  }
```

```
    // Method to display current balance
    public synchronized void displayBalance() {
        System.out.println("Final Current Balance: " + balance);
    }
}
// Deposit Thread
class DepositThread extends Thread {
    BankAccount account;

    DepositThread(BankAccount account) {
        this.account = account;
    }

    public void run() {
        account.deposit(500);
    }}

// Withdraw Thread
class WithdrawThread extends Thread {
    BankAccount account;

    WithdrawThread(BankAccount account) {
        this.account = account;
    }

    public void run() {
        account.withdraw(700);
    }}

// Main Class
public class BankSynchronization {
    public static void main(String[] args) {

        BankAccount account = new BankAccount();

        // Creating Threads
        DepositThread t1 = new DepositThread(account);
        WithdrawThread t2 = new WithdrawThread(account);

        // Starting Threads
        t1.start();
        t2.start();

        // Waiting for threads to complete
        try {
            t1.join();
```

```
            t2.join();
        } catch (InterruptedException e) {
            System.out.println(e);
        }
        // Display Final Balance
        account.displayBalance();
    }
}
```

OR

Write a program to demonstrate how the process tasks asynchronously in a Spring Boot application.

```
/* 1.go to start.spring.io
2.download file
3.open the file in intellij and run the application
4.go to src/main/java/com/example/demo/DemoApplication.java
5.create class AsyncService,AsyncController,AsyncDemoApplication */


/* ADD code in AsyncDemoApplication.java */
package com.example.firstdemo;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.scheduling.annotation.EnableAsync;
@SpringBootApplication
@EnableAsync
public class AsyncDemoApplication {
    public static void main (String[] args) {
        SpringApplication.run(AsyncDemoApplication.class, args);
    }}


/* Add code in AsyncService.java */
package com.example.firstdemo;
import org.springframework.scheduling.annotation.Async;
import org.springframework.stereotype.Service;
```

```
@Service
public class AsyncService {
   @Async
   public void processTask() {
      try {
         System.out.println("Task started on thread: " +
               Thread.currentThread().getName());
         Thread.sleep(5000); // simulate long process (5 seconds)
         System.out.println("Task completed on thread:" +
               Thread.currentThread().getName());
      } catch (InterruptedException e) {
         e.printStackTrace();
      } }}

/* Add code in AsyncController.java */
package com.example.firstdemo;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
@RestController
public class AsyncController {
   @Autowired
   private AsyncService asyncService;
   @GetMapping("/run")
   public String runTask() {
      asyncService.processTask();
      return "Task triggered successfully";
   }}
```

Slip 16

Create a hierarchy of Employee, Manager, Sales Manager, they should have the

following functionality:

a) Employee: Display name, date of birth and id of Employee.

b) Manager: Display all above information with salary drawn.

c) Sales Manager: Display all above information and commission if applicable.

```
import java.util.Scanner;
class Employee{
   int empId;
   String empName;
   String empDob;

   Employee(int empId, String empName, String empDob)   {
      this.empId = empId;
      this.empName = empName;
      this.empDob = empDob;
   }
   void displayEmployee(){
      System.out.println("\n===== EMPLOYEE DETAILS =====");
      System.out.println("Employee ID : " + empId);
      System.out.println("Employee Name : " + empName);
      System.out.println("Date of Birth : " + empDob);
   }}
class Manager extends Employee{
   double salary;
   Manager(int id, String name,String dob, double salary){
      super(id, name, dob);
      this.salary = salary;
   }
   void displayManager() {
      System.out.println("\n===== MANAGER DETAILS =====");
      System.out.println("Manager ID : " + empId);
      System.out.println("Manager Name : " + empName);
```

```
        System.out.println("Date of Birth : " + empDob);
        System.out.println("Salary : " + salary);
    }}
class SalesManager extends Manager{
    double commission;
    SalesManager(int id, String name, String dob, double salary, double commission)  {
        super(id, name, dob, salary);
        this.commission = commission;
    }
    void displaySalesManager() {
        System.out.println("\n===== SALES MANAGER DETAILS =====");
        System.out.println("Sales Manager ID : " + empId);
        System.out.println("Sales Manager Name : " + empName);
        System.out.println("Date of Birth : " + empDob);
        System.out.println("Salary : " + salary);
        System.out.println("Commission : " + commission);
    }}
public class Company{
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);

        System.out.println("\nEnter Employee Details");
        System.out.print("Enter Employee ID: ");
        int eid = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter Employee Name: ");
        String ename = sc.nextLine();
        System.out.print("Enter Employee DOB: ");
        String edob = sc.nextLine();
        System.out.println("\nEnter Manager Details");
```

```
System.out.print("Enter Manager ID: ");
int mid = sc.nextInt();
sc.nextLine();

System.out.print("Enter Manager Name: ");
String mname = sc.nextLine();

System.out.print("Enter Manager DOB: ");
String mdob = sc.nextLine();

System.out.print("Enter Manager Salary: ");
double msalary = sc.nextDouble();
sc.nextLine();
System.out.println("\nEnter Sales Manager Details");
System.out.print("Enter Sales Manager ID: ");
int sid = sc.nextInt();
sc.nextLine();

System.out.print("Enter Sales Manager Name: ");
String sname = sc.nextLine();
System.out.print("Enter Sales Manager DOB: ");
String sdob = sc.nextLine();
System.out.print("Enter Sales Manager Salary: ");
double ssalary = sc.nextDouble();
System.out.print("Enter Commission: ");
double commission = sc.nextDouble();

Employee e =new Employee(eid, ename, edob);
Manager m =new Manager(mid, mname,mdob, msalary);
SalesManager sm = new SalesManager(sid, sname,sdob, ssalary, commission);
```

```
        e.displayEmployee();
        m.displayManager();
        sm.displaySalesManager();
        sc.close();
    }}
```

OR

Write a program to implement patient information in a file and perform the file operations on it.

```
import java.io.*;
import java.util.*;
public class PatientFile{
    public static void main(String args[]) throws Exception{
        Scanner sc = new Scanner(System.in);
        int choice;
        do{
            System.out.println("\n===== PATIENT MENU =====");
            System.out.println("1. Add Patient");
            System.out.println("2. View Patients");
            System.out.println("3. Delete Patient");
            System.out.println("4. Exit");

            System.out.print("Enter Choice: ");
            choice = sc.nextInt();
            switch(choice)  {
                case 1:
                    FileWriter fw =new FileWriter("patient.txt", true);
                    sc.nextLine();
                    System.out.print("Enter Patient ID: ");
                    String id = sc.nextLine();

                    System.out.print("Enter Patient Name: ");
```

```
            String name = sc.nextLine();

            System.out.print("Enter Disease: ");
            String disease = sc.nextLine();
            fw.write(id + "," + name + "," +disease + "\n");
            fw.close();
            System.out.println("Patient Added Successfully");
            break;
        case 2:
            BufferedReader br =new BufferedReader(new FileReader("patient.txt"));

            String line;
            System.out.println("\n===== PATIENT RECORDS =====");
            while((line = br.readLine()) != null){
                String data[] = line.split(",");

                System.out.println("ID : " + data[0] +"  Name : " + data[1] +"  Disease : " +
data[2]);
            }
            br.close();
            break;
        case 3:
            sc.nextLine();
            System.out.print("Enter Patient ID to Delete: ");
            String deleteId =  sc.nextLine();
            File inputFile =new File("patient.txt");
            File tempFile =new File("temp.txt");
            BufferedReader reader =new BufferedReader(new FileReader(inputFile));
            BufferedWriter writer =new BufferedWriter(new FileWriter(tempFile));
            String currentLine;
```

```
            while((currentLine =
                  reader.readLine()) != null)
            {
               String data[] =currentLine.split(",");
               if(!data[0].equals(deleteId))
               {
                  writer.write(
                        currentLine + "\n");
               }}
            reader.close();
            writer.close();
            inputFile.delete();
            tempFile.renameTo(inputFile);
            System.out.println("Patient Deleted Successfully");
            break;
         case 4:
            System.out.println("Program Ended");
            break;
         default:
            System.out.println("Invalid Choice");
      }
   } while(choice != 4);
   sc.close();
}}
```

Slip 17

Implement a Java Program for creating a digital clock using AWT, Swing & Date class.

```
import java.awt.*;
import java.text.SimpleDateFormat;
import java.util.Date;
```

```
public class DigitalClock extends Frame implements Runnable{
   Label l;
   DigitalClock()
   {
      l = new Label();
      l.setFont(new Font("Arial", Font.BOLD, 30));
      add(l);
      setSize(400,150);
      setTitle("Digital Clock");
      setVisible(true);
      Thread t = new Thread(this);
      t.start();
   }

   public void run(){
      while(true)
      {
         Date d = new Date();
         SimpleDateFormat sdf =new SimpleDateFormat("HH:mm:ss");
         l.setText(sdf.format(d));
         try{
            Thread.sleep(1000);
         }
         catch(Exception e)
         {
         }
      }}
   public static void main(String args[]){
      new DigitalClock();
   }}
```

OR

Write a program to design an admission enquiry form using Swing.

/*Write a program to design an admission enquiry for MCA student form using Swing.*/

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class MCAAdmissionForm extends JFrame implements ActionListener
{
   JLabel l1, l2, l3, l4, l5, l6;
   JTextField t1, t2, t3;
   JRadioButton r1, r2;
   JComboBox c1;
   JButton b1;

   MCAAdmissionForm()
   {
      setTitle("MCA Admission Enquiry Form");

      l1 = new JLabel("Student Name:");
      l2 = new JLabel("Mobile Number:");
      l3 = new JLabel("Email ID:");
      l4 = new JLabel("Gender:");
      l5 = new JLabel("City:");
      l6 = new JLabel("Course:");

      t1 = new JTextField();
      t2 = new JTextField();
      t3 = new JTextField();
```

```
r1 = new JRadioButton("Male");
r2 = new JRadioButton("Female");

ButtonGroup bg = new ButtonGroup();
bg.add(r1);
bg.add(r2);

String course[] = {"MCA", "MBA", "MSc IT"};
c1 = new JComboBox(course);

b1 = new JButton("Submit");

setLayout(null);

l1.setBounds(50, 50, 120, 30);
t1.setBounds(180, 50, 150, 30);

l2.setBounds(50, 100, 120, 30);
t2.setBounds(180, 100, 150, 30);

l3.setBounds(50, 150, 120, 30);
t3.setBounds(180, 150, 150, 30);

l4.setBounds(50, 200, 120, 30);
r1.setBounds(180, 200, 80, 30);
r2.setBounds(260, 200, 80, 30);

l5.setBounds(50, 250, 120, 30);

String city[] = {"Mumbai", "Pune", "Nashik"};
```

```
JComboBox cityBox = new JComboBox(city);
cityBox.setBounds(180, 250, 150, 30);

l6.setBounds(50, 300, 120, 30);
c1.setBounds(180, 300, 150, 30);

b1.setBounds(150, 370, 100, 30);

add(l1);
add(t1);

add(l2);
add(t2);

add(l3);
add(t3);

add(l4);
add(r1);
add(r2);

add(l5);
add(cityBox);

add(l6);
add(c1);

add(b1);

b1.addActionListener(this);
```

```
        setSize(450, 500);
        setVisible(true);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }

    public void actionPerformed(ActionEvent e)
    {
        JOptionPane.showMessageDialog(this,
                "Admission Enquiry Submitted Successfully");
    }

    public static void main(String args[])
    {
        new MCAAdmissionForm();
    }
}
```

Slip 18

Develop a Java-based library management application that demonstrates OOP principles and uses exception handling to handle cases like issuing books, returning books, etc.

```
import java.util.Scanner;
// User Defined Exception
class BookException extends Exception
{
    BookException(String msg)
    {
        super(msg);
    }}
// Book Class
class Book
```

```
{
    int id;
    String name;
    boolean available = true;
    // Add Book
    void addBook(int i, String n)
    {
        id = i;
        name = n;
        System.out.println("Book Added Successfully");
    }
    // Issue Book
    void issueBook() throws BookException
    {
        if(available) {
            available = false;
            System.out.println("Book Issued Successfully");
        }
        else  {
            throw new BookException("Book Already Issued");
        }}
    // Return Book
    void returnBook() throws BookException
    {
        if(!available){
            available = true;
            System.out.println("Book Returned Successfully");
        }
        else
        {
            throw new BookException("Book Already Available");
```

```
    }}
  // Display Book
  void displayBook()
  {
    System.out.println("\nBook ID : " + id);
    System.out.println("Book Name : " + name);
    if(available){
      System.out.println("Status : Available");
    }
    else{
      System.out.println("Status : Issued");
    }
  }}
public class LibraryManagement{
  public static void main(String args[])
  {
    Scanner sc = new Scanner(System.in);
    Book b = new Book();
    int choice;
    do{
      System.out.println("\n===== LIBRARY MENU =====");
      System.out.println("1. Add Book");
      System.out.println("2. Issue Book");
      System.out.println("3. Return Book");
      System.out.println("4. Display Book");
      System.out.println("5. Exit");
      System.out.print("Enter Choice: ");
      choice = sc.nextInt();
      try{
        switch(choice)
        {
```

```
            case 1:
               System.out.print("Enter Book ID: ");
               int id = sc.nextInt();
               sc.nextLine();
               System.out.print("Enter Book Name: ");
               String name = sc.nextLine();b.addBook(id, name);
               break;
            case 2:b.issueBook();
               break;
            case 3:b.returnBook();
               break;
            case 4:b.displayBook();
               break;
            case 5:System.out.println("Program Ended");
               break;
            default:
               System.out.println("Invalid Choice");
         } }
      catch(BookException e) {
         System.out.println(e.getMessage());
      }
   } while(choice != 5);
   sc.close();
}}
```

OR

Write a program to demonstrate how the process tasks asynchronously in a Spring Boot application.

```
/* 1.go to start.spring.io
```

2.download file

3.open the file in intellij and run the application

4.go to src/main/java/com/example/demo/DemoApplication.java

5.create class AsyncService,AsyncController,AsyncDemoApplication */

```
/* ADD code in AsyncDemoApplication.java */
package com.example.firstdemo;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.scheduling.annotation.EnableAsync;
@SpringBootApplication
@EnableAsync
public class AsyncDemoApplication {
   public static void main (String[] args) {
      SpringApplication.run(AsyncDemoApplication.class, args);
   }}

/* Add code in AsyncService.java */
package com.example.firstdemo;
import org.springframework.scheduling.annotation.Async;
import org.springframework.stereotype.Service;
@Service
public class AsyncService {
   @Async
   public void processTask() {
      try {
         System.out.println("Task started on thread: " +
               Thread.currentThread().getName());
         Thread.sleep(5000); // simulate long process (5 seconds)
         System.out.println("Task completed on thread:" +
               Thread.currentThread().getName());
```

```
        } catch (InterruptedException e) {
            e.printStackTrace();
        } }}

/* Add code in AsyncController.java */
package com.example.firstdemo;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
@RestController
public class AsyncController {
    @Autowired
    private AsyncService asyncService;
    @GetMapping("/run")
    public String runTask() {
        asyncService.processTask();
        return "Task triggered successfully";
    }}
```

Slip 19

Write a program to accept roll no. Marks from users. Throw user defined exception Marks out of Bound if marks are 100.

```
package Slip12_1;
import java.util.Scanner;
class MarksOutOfBoundException extends Exception
{
    MarksOutOfBoundException(String msg){
        super(msg);
    }
}
public class  Student
{
```

```
public static void main(String args[])
{
   Scanner sc = new Scanner(System.in);
   try
   {
      System.out.print("Enter Roll Number: ");
      int rollNo = sc.nextInt();
      sc.nextLine();

      System.out.print("Enter Marks: ");
      int marks = sc.nextInt();
      sc.nextLine();
      if(marks > 100)
      {
         throw new MarksOutOfBoundException(
               "Marks Out Of Bound");
      }
      System.out.println("Roll No: " + rollNo);
      System.out.println("Marks: " + marks);
   }
   catch(MarksOutOfBoundException e)
   {
      System.out.println(e);
   }
   sc.close();
}}
```

OR

Create a Java application that connects to a MySQL database to manage employee information. Implement functionalities like adding, updating, deleting, and viewing employee records using JDBC. Use prepared statements to prevent SQL injection and handle exceptions gracefully.

```
import java.sql.*;
import java.util.Scanner;

public class EmployeeCRUD
{
   static final String URL ="jdbc:mysql://localhost:3306/slips";
   static final String USER = "root";
   static final String PASSWORD = "Shubham.k";

   public static void main(String args[])
   {
      Scanner sc = new Scanner(System.in);
      try
      {
         Connection con =DriverManager.getConnection(URL, USER, PASSWORD);
         int choice;
         do
         {
            System.out.println("\n===== EMPLOYEE MENU =====");
            System.out.println("1. Add Employee");
            System.out.println("2. Update Employee");
            System.out.println("3. Delete Employee");
            System.out.println("4. View Employees");
            System.out.println("5. Exit");

            System.out.print("Enter Choice: ");
            choice = sc.nextInt();
            switch(choice)
            {
               // Add Employee
               case 1:
```

```
    System.out.print("Enter Employee ID: ");
    int id = sc.nextInt();
    sc.nextLine();
    System.out.print("Enter Employee Name: ");
    String name = sc.nextLine();
    System.out.print("Enter Salary: ");
    double salary = sc.nextDouble();
    String insertQuery = "INSERT INTO slip19_2 VALUES(?,?,?)";
    PreparedStatement ps1 =con.prepareStatement(insertQuery);

    ps1.setInt(1, id);
    ps1.setString(2, name);
    ps1.setDouble(3, salary);
    ps1.executeUpdate();
    System.out.println("Employee Added Successfully");
    break;

// Update Employee
case 2:
    System.out.print("Enter Employee ID: ");
    int updateId = sc.nextInt();
    sc.nextLine();
    System.out.print("Enter New Name: ");
    String newName = sc.nextLine();
    String updateQuery = "UPDATE slip19_2 SET name=? WHERE id=?";
    PreparedStatement ps2 =con.prepareStatement(updateQuery);
    ps2.setString(1, newName);
    ps2.setInt(2, updateId);
    ps2.executeUpdate();

    System.out.println("Employee Updated Successfully");
```

```
    break;
// Delete Employee
case 3:
    System.out.print("Enter Employee ID: ");
    int deleteId = sc.nextInt();
    String deleteQuery = "DELETE FROM slip19_2 WHERE id=?";

    PreparedStatement ps3 =con.prepareStatement(deleteQuery);
    ps3.setInt(1, deleteId);
    ps3.executeUpdate();
    System.out.println("Employee Deleted Successfully");
    break;
// View Employees
case 4:
    String selectQuery ="SELECT * FROM slip19_2";

    PreparedStatement ps4 = con.prepareStatement(selectQuery);
    ResultSet rs = ps4.executeQuery();
    System.out.println("\n===== EMPLOYEE RECORDS =====");
    while(rs.next())
    {
       System.out.println(
            rs.getInt("id") + "  " +
                rs.getString("name") + "  " +
                rs.getDouble("salary"));
    }
    break;
case 5:
    System.out.println("Program Ended");
    break;
default:
```

```
                System.out.println("Invalid Choice");
            }
        } while(choice != 5);
        con.close();
    }
    catch(Exception e)
    {
        System.out.println(e);
    }
    sc.close();
}}
```

Slip 20

Implement a Java program to manage bank accounts using object-oriented programming (OOP) concepts like classes, inheritance, encapsulation, and polymorphism. Include features like account creation, deposit, withdrawal, and balance inquiry.

```
package Slip20_1;
import java.util.Scanner;
class Account
{
    int accNo;
    String name;
    double balance;
    // Create Account
    void createAccount(int no, String n, double bal)
    {
        accNo = no;
        name = n;
        balance = bal;
        System.out.println("Account Created Successfully");
```

```
    }
    // Deposit
    void deposit(double amount)
    {
        balance = balance + amount;
        System.out.println("Amount Deposited");
    }
    // Withdraw
    void withdraw(double amount)
    {
        if(balance >= amount)
        {
            balance = balance - amount;
            System.out.println("Amount Withdrawn");
        }
        else
        {
            System.out.println("Insufficient Balance");
        }
    }
    // Balance Inquiry
    void checkBalance()
    {
        System.out.println("\nAccount No : " + accNo);
        System.out.println("Name : " + name);
        System.out.println("Balance : " + balance);
    }
}
// Derived Class
class SavingsAccount extends Account
{
```

```
    void checkBalance()   // Polymorphism
    {
        System.out.println("\nSavings Account Balance : " + balance);
    }
}
public class BankSystem
{
    public static void main(String args[])
    {
        Scanner sc = new Scanner(System.in);
        SavingsAccount a = new SavingsAccount();
        int choice;
        do
        {
            System.out.println("\n===== BANK MENU =====");
            System.out.println("1. Create Account");
            System.out.println("2. Deposit");
            System.out.println("3. Withdraw");
            System.out.println("4. Check Balance");
            System.out.println("5. Exit");
            System.out.print("Enter Choice: ");
            choice = sc.nextInt();
            switch(choice)
            {
                case 1:
                    System.out.print("Enter Account Number: ");
                    int no = sc.nextInt();
                    sc.nextLine();

                    System.out.print("Enter Name: ");
                    String name = sc.nextLine();
```

```
            System.out.print("Enter Initial Balance: ");
            double bal = sc.nextDouble();
            a.createAccount(no, name, bal);
            break;
        case 2:
            System.out.print("Enter Deposit Amount: ");
            double dep = sc.nextDouble();
            a.deposit(dep);
            break;
        case 3:
            System.out.print("Enter Withdraw Amount: ");
            double wd = sc.nextDouble();
            a.withdraw(wd);
            break;
        case 4:
            a.checkBalance();
            break;
        case 5:
            System.out.println("Program Ended");
            break;
        default:
            System.out.println("Invalid Choice");
      }
    } while(choice != 5);
    sc.close();
}}
```

OR

Write a program of servlet to demonstrate GET and POST Methods with suitable example using simulation.

/*Write a program of servlet to demonstrate GET and POST Methods with suitable

```
example using simulation.*/


import java.util.Scanner;

public class RequestSimulator {

    // Simulating GET method
    public static void doGet(String name, String age) {
        System.out.println("\n--- GET Method Execution ---");
        System.out.println("URL: http://localhost:8080/student?name=" + name +
                "&age=" + age);

        System.out.println("Data received using GET method:");
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }

    // Simulating POST method
    public static void doPost(String name, String age) {
        System.out.println("\n--- POST Method Execution ---");
        System.out.println("URL: http://localhost:8080/student");
        System.out.println("(Data is sent in request body, not visible in URL)");

        System.out.println("Data received using POST method:");
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }

    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);

        System.out.println("=== Servlet GET/POST Simulation ===");

        // Taking input
        System.out.print("Enter Name: ");
        String name = sc.nextLine();

        System.out.print("Enter Age: ");
        String age = sc.nextLine();

        // Choosing method
        System.out.println("\nChoose Method:");
        System.out.println("1. GET");
        System.out.println("2. POST");
        System.out.print("Enter choice: ");

        int choice = sc.nextInt();

        // Calling respective method
        if (choice == 1) {
            doGet(name, age);
        } else if (choice == 2) {
            doPost(name, age);
        } else {
            System.out.println("Invalid choice!");
        }

        sc.close();
    }
}
```